ResultIQ

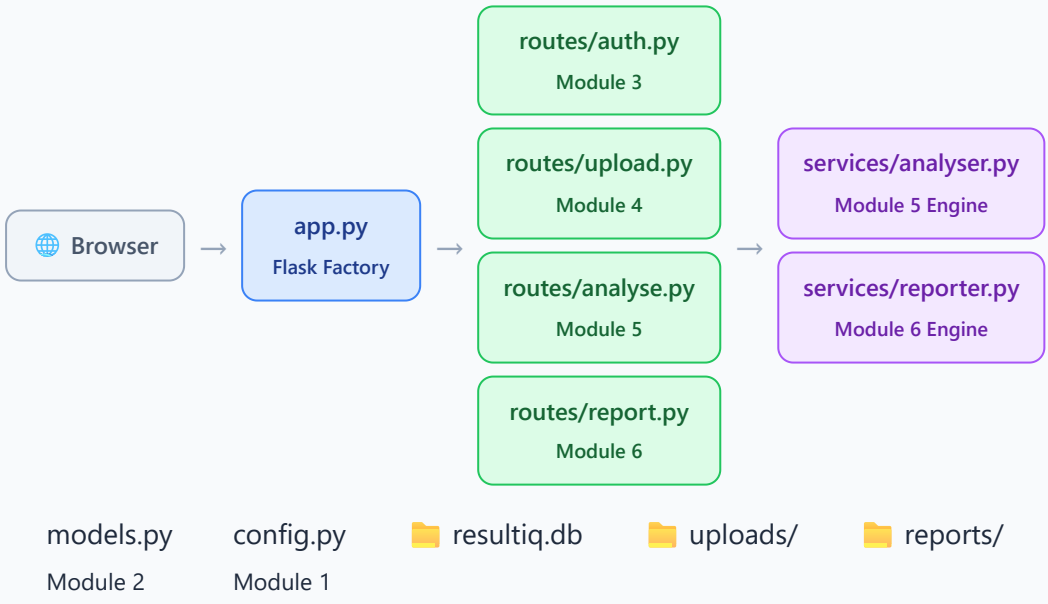
Complete Module Guide

Intelligent Excel Mark-Sheet Analysis & Report Generation Platform

What is ResultIQ? — A Flask web application that lets teachers upload Excel mark sheets, automatically detect which columns contain student marks, and generate professional result-analysis reports — all without writing a single formula.

High-Level Architecture

SYSTEM ARCHITECTURE



Module 1 — `config.py` (Application Configuration)

Purpose: Central place for all configuration settings — no magic strings scattered across files.

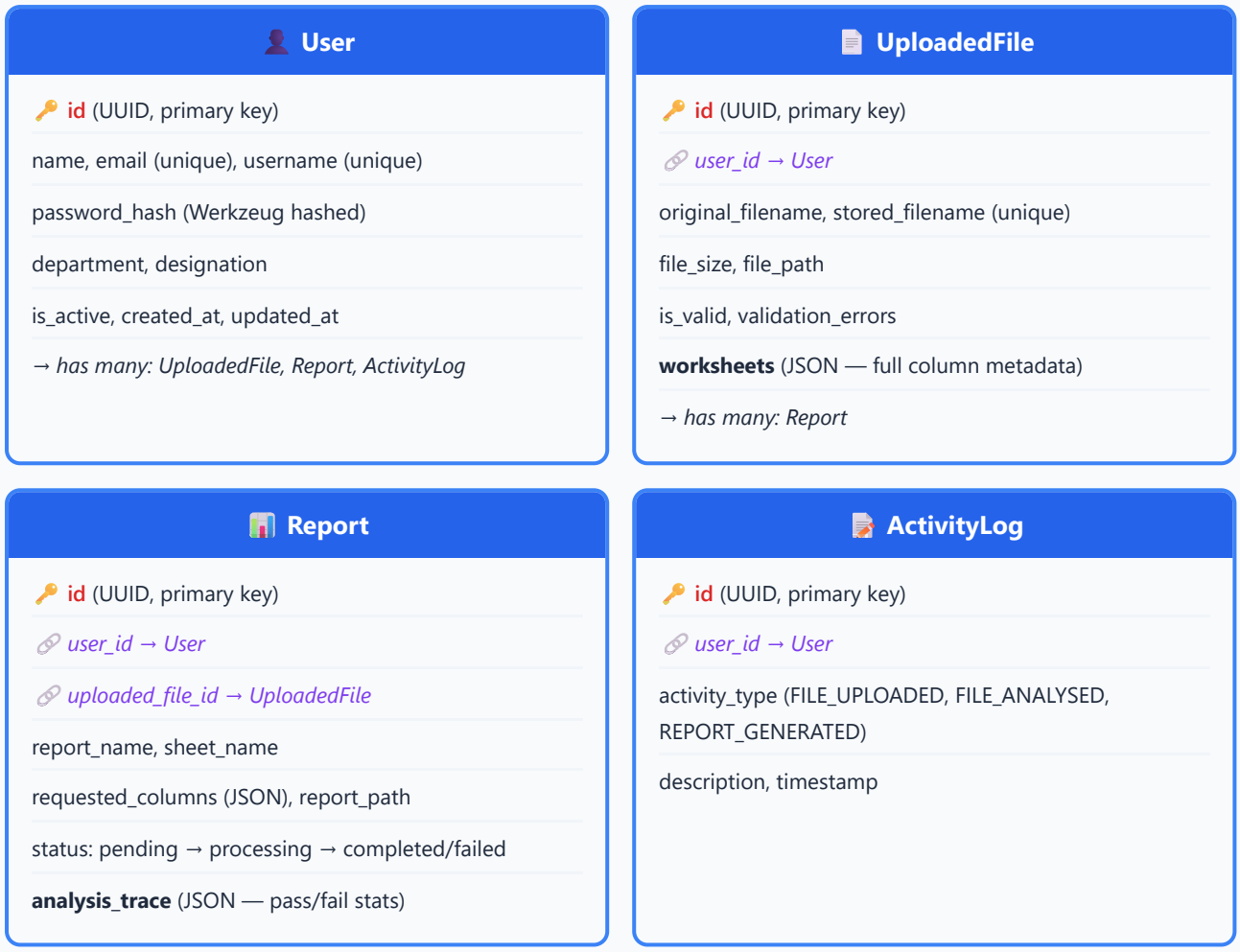
Setting	Details
Secret Key	For session signing — reads `SECRET_KEY` env var, falls back to a dev default
Session Lifetime	30-day persistent sessions so users stay logged in
Upload & Report Folders	`uploads/` and `reports/` directories, auto-created at startup
Max Upload Size	16 MB cap to prevent oversized files
Database URI	SQLite file `resultiq.db` (or `DATABASE_URL` env var for production)
Config Profiles	`DevelopmentConfig` (debug on), `TestingConfig` (separate test DB), `ProductionConfig`

Key takeaway: You pick a profile name (`'development'` , `'testing'` , `'production'`) and the app loads the matching settings automatically.

Module 2 — `models.py` (Database Models)

Purpose: Defines the four SQLAlchemy ORM tables that store *everything* the app needs to remember.

ENTITY RELATIONSHIP DIAGRAM



Application Factory — `app.py` (Wiring Module)

Purpose: The entry point that wires everything together using Flask's application factory pattern.

What it does step by step:

1. Creates a `Flask` app and loads config from `config.py`
2. Creates `uploads/` and `reports/` directories if missing
3. Initialises **SQLAlchemy** (database ORM) and **Flask-Login** (session auth)
4. Registers the `user_loader` callback so Flask-Login can look up users by ID
5. Adds a custom Jinja2 `enumerate` filter for templates
6. Imports and registers **4 Blueprints**: `auth_bp`, `upload_bp`, `analyse_bp`, `report_bp`
7. Defines the root route `/` → renders `index.html` (landing page)
8. Registers error handlers for **404** (Not Found) and **500** (Server Error)
9. When run directly (`python app.py`), creates all DB tables and starts the dev server

Module 3 — `routes/auth.py` (Authentication)

Purpose: Handles user registration, login, logout, and the dashboard.

Route	Method	What it does
`/register`	GET / POST	Shows registration form. On POST: validates all fields, enforces **strong password rules** (≥8 chars, uppercase, lowercase, digit, special char, no spaces), checks for duplicate email/username, creates the user.
`/login`	GET / POST	Shows login form. Accepts **email or username**. Strips accidental trailing spaces from passwords. Sets a 30-day persistent session. Redirects to `?next=` URL if present.
`/logout`	GET	Clears session, logs out, redirects to home.
`/dashboard`	GET	Shows the user's dashboard with their **5 most recent reports**.

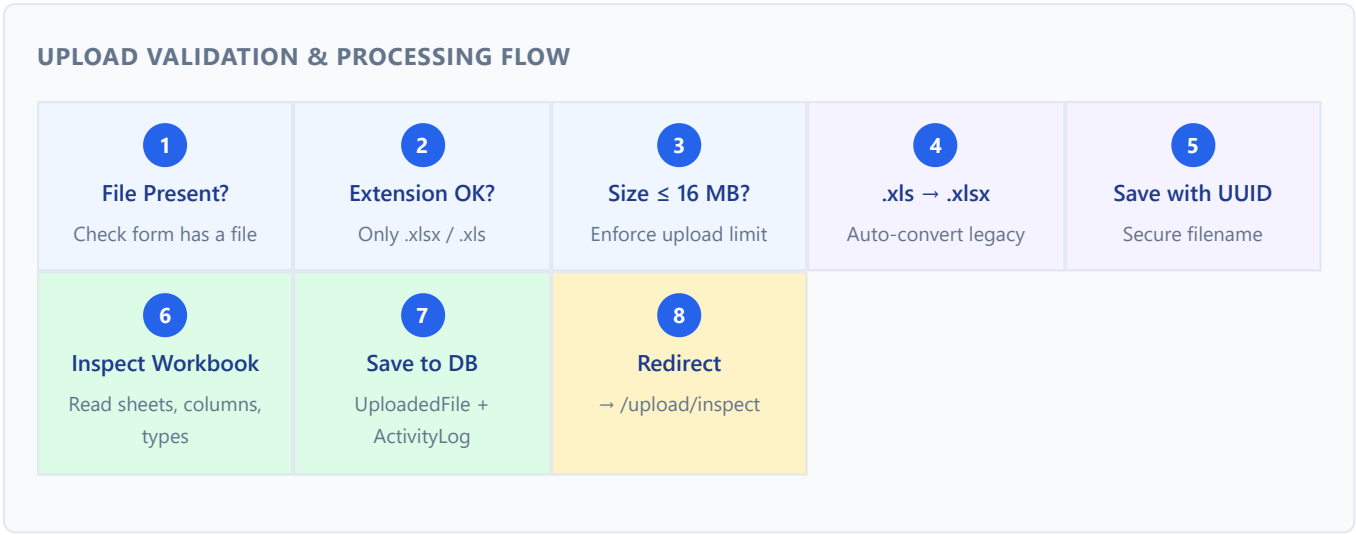
> **Security:** Passwords are stored as Werkzeug-hashed values (never plaintext). Login gives distinct error messages for "account not found" vs "wrong password". All routes except `/register` and `/login` require `@login_required`.

Module 4 — `routes/upload.py` (File Upload & Inspection)

Purpose: Lets users upload `.xlsx` / `.xls` mark sheets and immediately inspects their structure.

Route	Method	What it does
`/upload`	GET	Shows upload form + list of 5 recently uploaded files
`/upload`	POST	Receives the Excel file, validates, saves, inspects, stores metadata
`/upload/inspect/<file_id>`	GET	Displays the inspection results (sheets, columns, data types, samples)

Upload Pipeline (POST)



`inspect_workbook()` function

- Opens the Excel file with pandas (`openpyxl` for `.xlsx` , `xlrd` for `.xls`)
- For each worksheet: reads columns, infers data types (Integer, Decimal, Text, Date/Time, Boolean), grabs 3 sample values
- Skips unnamed/empty columns
- Returns a clean JSON structure stored in `UploadedFile.worksheets`

Module 5 — `services/analyser.py` (Intelligent Column Classifier)

> **This is the brain of ResultIQ.** It automatically classifies every column as one of: `marks`, `identity`, `metadata`, `category`, `empty`, or `text`.
>
> **Zero Flask imports** — pure Python + pandas, making it independently testable.

Classification Logic — `classify_column()`

The classifier uses a **13-step decision cascade**:

Step	Check	Result	Example Columns Caught
1	Datetime type or name	`metadata`	DOB, Timestamp
2	Identity name patterns (regex)	`identity`	Sl. No, Roll No, USN, Student Name, Email, Aadhaar
3	Excluded summary columns (regex)	`metadata` / `category`	Total Marks, Percentage, CGPA, Rank, Grade, Attendance, Result
4	Unnamed columns	`metadata`	Unnamed: 0 (from multi-row headers)
5	Long names (>40 chars)	`metadata`	Free-form description columns
6	Nearly empty (<5% filled)	`empty`	Placeholder columns
7	Large numbers (median >999)	`identity`	Registration numbers, phone numbers
8	Rating / Likert guard	`metadata`	Survey ratings 1–5, year columns
9	Subject name match + numeric data	`marks`	Mathematics, Physics, 24CS301, CIT-1, Quiz-2
10	Content-driven: ≥50% numeric, ≥75% coverage, median ≤200	`marks`	Unnamed but clearly numeric score columns
11	High uniqueness non-numeric	`identity`	Student IDs with unique text
12	Low cardinality text	`category`	Section (A/B/C), Gender
13	Default fallback	`text`	Everything else

Header Detection — `_detect_header_row()`

- Scans first 35 rows, scores each candidate based on keyword matches (`Name`, `Roll No`, `Marks`, subject codes) and whether the next row has data

- Picks the highest-scoring row as the real header — handles messy sheets with title rows, blank rows, or merged cells above the actual table

Quality Metrics — `_build_quality()`

For every column, computes: total rows, filled count, missing count, numeric count, absent-token count, text count, unique values.

Route Layer — `routes/analyse.py`

Route	Method	What it does
`/analyse/<file_id>`	GET	Runs `analyse_workbook()` on the uploaded file, saves the enriched worksheets JSON back to the DB, renders the results page with classification badges and a "Generate Report" button.

Module 6 — `services/reporter.py` (Report Generation Engine)

> **Zero Flask imports** — pure Python + pandas + openpyxl. Fully testable in isolation.

What `generate_report()` does

REPORT GENERATION PIPELINE

1	2	3	4	5
Read Marks Cols	**Read Source Excel**	**Fix Headers**	**Filter Rows**	**Compute Results**
From classification JSON	Using detected header row	Multi-row / Unnamed cols	Skip footers, signatures	Total, Avg, %, Pass/Fail

6	7	8
Build Stats	**Clone Workbook**	**Append Analysis**
Summary statistics	Preserve source layout	Formulas + styling

Computed Columns Added Per Student

Column	Formula / Logic
Total Marks	Sum of all marks (absent → 0)
Subjects Appeared	Count of subjects with a valid mark
Average	Mean of appeared subjects' marks
Percentage	(Total / max possible) × 100
Result	**PASS** if overall ≥40% AND every subject ≥50%; else **FAIL**

Per-Subject Analysis Block (Appended Below Student Data)

Row Label	What It Shows
appeared	Students who took this subject
passed	Students who scored ≥50 in this subject
% passed	Pass percentage for the subject
below 50%	Count in score bracket
50%–60%	Count in score bracket
61%–70%	Count in score bracket
71%–80%	Count in score bracket
81%–90%	Count in score bracket
91%–100%	Count in score bracket

Excel Styling Features

- Grey header row with bold Arial font and thick borders
- Red highlight for marks < 50 (fail)
- **Grey highlight** for absent students (AA, Abs, etc.)
- Native Excel conditional formatting rules
- Auto-sized column widths (names wider, marks centered)
- Monkeypatched openpyxl writer to output cached formula values in `<v>` tags

Absent Token Recognition (20+ markers)

`AA`, `Ab`, `Abs`, `Absent`, `NA`, `N/A`, `-`, `--`, `Nil`, `None`, `Withheld`, `Medical`, `Detained`, `Exempted`, `OD`, `On Duty`, `Leave`, and more.

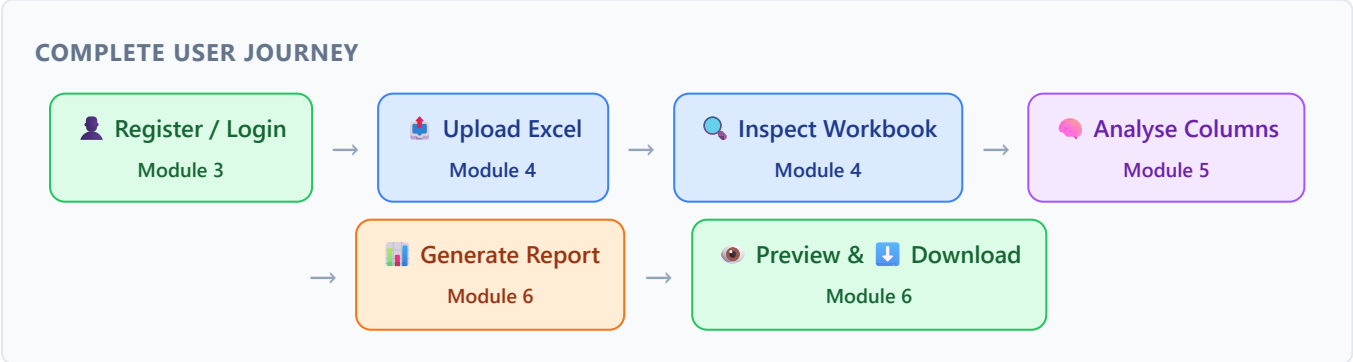
Route Layer — `routes/report.py`

Route	Method	What it does
`/report/generate`	POST	Validates inputs, creates a Report record (status= `processing`), calls `generate_report()`, updates status to `completed` or `failed`, redirects to preview
`/report/<id>`	GET	Reads the generated `.xlsx`, displays a data table preview with summary stats in the browser
`/report/<id>/download`	GET	Streams the `.xlsx` file as a download, records the download timestamp
`/reports`	GET	Lists all reports for the current user, ordered by most recent first

Templates & Static Assets

Path	Purpose
`templates/base.html`	Base layout with navigation bar, flash messages, Bootstrap CSS/JS
`templates/index.html`	Landing / home page
`templates/dashboard.html`	User dashboard with recent reports
`templates/auth/login.html`	Login form (email or username)
`templates/auth/register.html`	Registration form with password strength rules
`templates/upload/index.html`	File upload form + recent uploads list
`templates/upload/inspect.html`	Workbook inspection results (sheets, columns, samples)
`templates/analyse/results.html`	Column classification results + Generate Report button
`templates/report/preview.html`	Report preview with stats + data table + download button
`templates/report/history.html`	All past reports for the user
`templates/404.html`	Not Found error page
`templates/500.html`	Internal Server Error page
`static/css/style.css`	Custom CSS styles
`static/js/main.js`	Minimal JS foundation

End-to-End User Flow



1. **Register / Login** — Create an account or sign in with email/username
2. **Upload** — Drop an `.xlsx` / `.xls` mark sheet (auto-converts legacy format)
3. **Inspect** — See all sheets, columns, data types, and sample values at a glance

4. **Analyse** — Intelligent classifier tags each column (marks / identity / metadata / category) using 13-step logic
5. **Generate** — Engine computes Total, Average, Percentage, Pass/Fail per student + per-subject analysis
6. **Preview & Download** — See the full results table in-browser, then download a professionally styled Excel file

Tech Stack

Layer	Technology
Backend Framework	**Flask 3.0** (Python)
Database	**SQLite** via Flask-SQLAlchemy
Authentication	**Flask-Login** + Werkzeug password hashing
Excel Reading	**pandas** + **openpyxl** (.xlsx) + **xlrd** (.xls)
Excel Writing	**openpyxl** with custom monkeypatched writer for formula caching
Frontend	Jinja2 templates + Bootstrap + minimal custom CSS/JS
Configuration	Python config classes + environment variables